中南林业科技大学

编译原理实验报告

课程名称：	编译原理
专业班级：	软工 3 班
姓　　名：	李佳鹏
学　　号：	20245428

完成日期：2026 年 11 月

目录

实验二　LL（1）分析法 1
一、　实验目的 1
二、　实验内容 1
三、　实验理论依据 4
四、　测试数据及运行结果展示 9

实验二　LL（1）分析法

一、 实验目的

根据给定的算术表达式文法，编制并调试一个 LL（1）分析程序，使其能够对任意输入的符号串进行分析，判断该串是否为该文法的句子，并把分析过程中分析栈、剩余输入串与所用产生式的变化逐行输出，以加深对预测分析法的理解。

通过本次实验，掌握预测分析法的完整设计流程：消除文法的左递归，计算非终结符的 FIRST 集与 FOLLOW 集，求各产生式的 SELECT 集并据此判定文法是否为 LL（1）文法，构造预测分析表，最后设计出以“分析栈＋预测分析表”为核心、无需回溯的自上而下分析程序。

二、 实验内容

（1）给出待分析的算术表达式文法。文法共 8 条产生式，见表 1。其中 E、G、T、S、F 为非终结符，i、+、*、(、)、# 为终结符，# 同时用作输入串的结束符。

表 1　算术表达式文法的产生式

序号	产生式	说　明
（1）	E → TG	表达式由项 T 与剩余部分 G 构成
（2）	G → +TG	处理加号，继续读入一个项
（3）	G → ε	表达式结束，没有更多的加减运算
（4）	T → FS	项由因子 F 与剩余部分 S 构成
（5）	S → *FS	处理乘号，继续读入一个因子
（6）	S → ε	项结束，没有更多的乘法运算
（7）	F → (E)	带括号的表达式
（8）	F → i	运算对象（标识符或常数）

（2）消除文法的左递归。分析给定文法是否含有直接左递归或间接左递归；对含左递归的表达式文法，按消除左递归的一般方法进行改写，并说明改写结果与表 1 给出的文法完全一致，其中 G、S 即 E′、T′。本程序并不在运行期消除左递归，而是把改写后的 8 条产生式直接写进程序，用结构体 type 表示一条产生式，左部存放在 origin 中，右部存放在字符数组 array 中，length 记录右部长度。

```
class type {                  /* 一条产生式 */
public:
    char origin;              /* 左部非终结符 */
    char array[5];            /* 右部字符 */
    int  length;              /* 右部长度 */
};
type e,t,g,g1,s,s1,f,f1;      /* 8 个对象，一一对应表 1 的 8 条产生式 */
type C[10][10];               /* 预测分析表 M，行取非终结符，列取终结符 */
```

main 函数开头把 8 条产生式逐条送入对象，ε 产生式没有右部字符，用字符 '^' 顶替，这样查表时不必再区分“空右部”与“右部只有 ε”两种情形（源程序 49～78 行）：

```
e.origin='E';  strcpy(e.array,"TG");  e.length=2;    /* E -> TG  */
g.origin='G';  strcpy(g.array,"+TG"); g.length=3;    /* G -> +TG */
g1.origin='G'; g1.array[0]='^';       g1.length=1;   /* G -> ε   */
t.origin='T';  strcpy(t.array,"FS");  t.length=2;    /* T -> FS  */
s.origin='S';  strcpy(s.array,"*FS"); s.length=3;    /* S -> *FS */
s1.origin='S'; s1.array[0]='^';       s1.length=1;   /* S -> ε   */
f.origin='F';  strcpy(f.array,"(E)"); f.length=3;    /* F -> (E) */
f1.origin='F'; f1.array[0]='i';       f1.length=1;   /* F -> i   */
```

（3）计算每个非终结符的 FIRST 集与 FOLLOW 集，结果见表 2。

（4）求每条产生式的 SELECT 集，按“同一非终结符的任意两条候选产生式的 SELECT 集不相交”判定该文法是否为 LL（1）文法，结果见表 3。

（5）按 SELECT 集填写预测分析表，见表 4。表中行对应非终结符，列对应终结符，空白表示出错。在程序里这张表就是二维数组 C[5][6]：行下标依次对应非终结符 E、G、T、S、F（数组 v2），列下标依次对应终结符 i、+、*、(、)、#（数组 v1）。填表分两步，先把全部表项的 origin 置成 'N' 表示出错，再按表 4 把产生式对象赋到对应位置，查表时取到的对象就是该步要用的产生式（源程序 80～89 行）：

```
for (m=0;m<=4;m++)     /* 第一步：30 个表项全部置空 */
    for (n=0;n<=5;n++)
        C[m][n].origin='N';

C[0][0]=e;  C[0][3]=e;                        /* E：i、( */
C[1][1]=g;  C[1][4]=g1; C[1][5]=g1;           /* G：+、)、# */
C[2][0]=t;  C[2][3]=t;                        /* T：i、( */
C[3][1]=s1; C[3][2]=s;  C[3][4]=C[3][5]=s1;   /* S：+、*、)、# */
C[4][0]=f1; C[4][3]=f;                        /* F：i、( */
```

（6）用 C++ 实现表驱动的分析程序：预测分析表用二维数组存放，分析栈用字符数组模拟，对输入串逐符号分析；接受时输出 acc，出错时指出出错的栈顶符号。分别用正确串 i+i*i# 与错误串 i+*i# 运行，记录分析过程（表 5 与表 6）。程序用几个一维数组分别存放分析栈、剩余输入串和符号表，读入阶段逐字符检查合法性，读完后把 '#' 与开始符号 'E' 压栈（源程序 7～12 行、92～105 行）：

```
char A[20];                              /* 分析栈 */
char B[20];                              /* 剩余输入串 */
char v1[20]={'i','+','*','(',')','#'};   /* 终结符表 */
char v2[20]={'E','G','T','S','F'};       /* 非终结符表 */
int  j=0,b=0,top=0,l;                    /* b：当前符号下标，l：串长 */

do {                                     /* 逐字符读入，遇 '#' 结束 */
    cin>>ch;
    if ((ch!='i')&&(ch!='+')&&(ch!='*')&&
        (ch!='(')&&(ch!=')')&&(ch!='#'))
    {   cout<<"输入串中有非法字符\n"; exit(1);   }
    B[j]=ch;  j++;
} while (ch!='#');
l=j;  ch=B[0];                           /* 串长，当前分析字符 */
A[top]='#';  A[++top]='E';               /* '#'、'E' 进栈 */
```

每一步都要按“步骤、分析栈、剩余字符、所用产生式”四列输出，分析栈与剩余串各由一个函数负责打印，print1 先补 b 个空格再输出，使当前符号在列上对齐（源程序 25～41 行）：

```
void print()                /* 输出分析栈 A[0..top+1] */
{
    int a;
    for (a=0;a<=top+1;a++) cout<<A[a];
    cout<<"\t\t";
}

void print1()               /* 输出剩余串，前补 b 个空格对齐 */
{
    int j;
    for (j=0;j<b;j++) cout<<" ";
    for (j=b;j<=l;j++) cout<<B[j];
    cout<<"\t\t\t";
}
```

主控循环是程序的核心：每轮先从栈中弹出栈顶符号 x，再按 x 是否属于终结符表 v1 分成两个分支。x 为终结符时进入 if 分支，先判断是否已读到结束符 #，是则接受并输出 acc，否则比较 x 与当前输入符号 ch：相同则匹配成功，输入指针后移一个符号；不同则报错：

```
do {
    x = A[top--];                                    /* 弹出栈顶符号 */
    for (j=0;j<=5;j++) if (x==v1[j]) { flag=1; break; } /* 是终结符? */
    if (flag==1) {                                   /* ---- 终结符 ---- */
        if (x=='#') { finish=1; cout<<"acc!"; exit(1); } /* 接受 */
        if (x==ch)  { print(); print1(); cout<<"匹 配"<<ch;
                      ch = B[++b]; flag=0; }             /* 匹配 */
        else       { print(); print1(); cout<<"出 错"<<ch; exit(1); }
    } else {                                         /* ---- 非终结符 ---- */
```

x 为非终结符时走 else 分支：先按 x 在 v2 中的位置定行号、按 ch 在 v1 中的位置定列号，取出表项 C[m][n]；表项非空就把该产生式右部逆序压栈，右部若为 '^' 则只退栈不入栈；表项为空（origin 为 'N'）则报错。循环直到 finish 置 1 为止：

```
        for (j=0;j<=4;j++) if (x==v2[j]) { m=j; break; } /* 行号 */
        for (j=0;j<=5;j++) if (ch==v1[j]) { n=j; break; } /* 列号 */
        cha = C[m][n];
        if (cha.origin!='N') {
            print(); print1(); cout<<cha.origin<<"->";
            for (j=cha.length-1;j>=0;j--)           /* 右部逆序压栈 */
                A[++top] = cha.array[j];
            if (A[top]=='^') top--;                 /* ε 不入栈 */
        } else { print(); print1(); cout<<"出 错"<<x; exit(1); }
    }
} while (finish==0);
```

实验环境：Windows 11，g++（MinGW-w64）8.1.0，C++11；源程序为单文件 LL1.cpp。

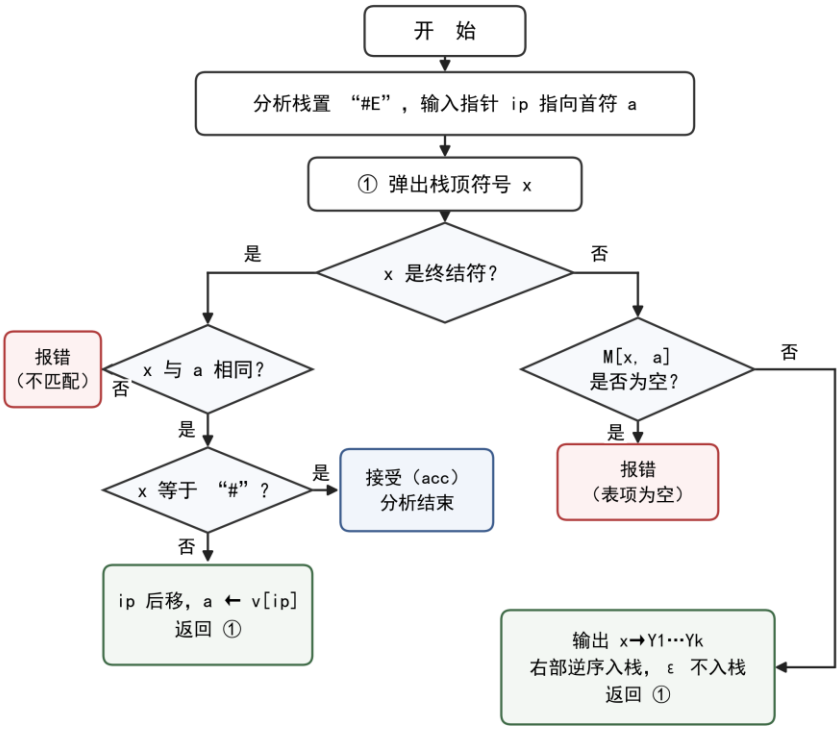
三、 实验理论依据

1. LL（1）分析法的基本模型

LL（1）分析法是一种自上而下的语法分析方法：从左到右扫描输入串（第一个 L），按最左推导的方向逐步展开（第二个 L），分析过程中只需向前查看一个输入符号（1）即可唯一确定下一步所用的产生式。正因为向前看一个符号就够，分析过程不需要回溯。

分析程序由三部分组成：一是分析栈，栈底先放结束符 #，再把文法的开始符号 E 压入栈；二是预测分析表 M，M[A, a] 给出“栈顶为非终结符 A、当前输入符号为 a”时应选用的产生式；三是输入缓冲区，存放待分析的符号串，末尾以 # 结束。程序在任一时刻的格局由“栈中内容、剩余输入串”完全确定，根据栈顶符号 x 与当前输入符号 a 执行以下四种动作之一。

① 若 x 为终结符且 x＝a，则匹配成功，弹出 x 并把输入指针后移一个符号；② 若 x 为终结符且 x≠a，则出错；③ 若 x 为非终结符，则查表 M[x, a]，若表项为产生式 x→Y1Y2…Yk，则弹出 x 并把 Yk…Y1 逆序压栈（ε 产生式不入栈），若表项为空则出错；④ 若栈顶与当前输入符号同为 #，则分析成功。总控程序流程如图 1 所示。



注：① 为循环入口；两条绿框分支执行后回到 ①，重新弹出栈顶符号。

图 1 LL（1）总控程序流程图

图 1 中，① 是循环入口。要注意的是产生式右部必须逆序入栈：因为栈是后进先出结构，只有让右部最左的符号位于栈顶，下一步才会优先展开它，从而保证分析过程始终按最左推导进行。

2．左递归的消除

若文法中含有形如 A⇒+Aα 的推导，则称该文法含有左递归。左递归会使自上而下的分析陷入无限循环：每展开一次 A，栈顶仍然是 A，永远无法与输入符号匹配。因此 LL（1）文法必须首先消除左递归。

对直接左递归，设产生式为 A→Aα1｜Aα2｜…｜Aαm｜β1｜β2｜…｜βn（βi 不以 A 开头），可改写为 A→β1A′ ｜β2A′ ｜…｜βnA′，A′ →α1A′ ｜α2A′ ｜…｜αmA′ ｜ε，改写前后产生的语言相同，但不再含左递归。对间接左递归，先用代入法把非终结符逐个展开，把间接左递归化为直接左递归后再按上式处理。

经典的算术表达式文法 E→E+T｜T，T→T*F｜F，F→(E)｜i 含有直接左递归，不能直接用于 LL（1）分析。对它施用上述改写，得 E→TE′，E′ →+TE′ ｜ε；T→FT′，T′ →*FT′ ｜ε；F→(E)｜i。令 E′ 记作 G、T′ 记作 S，即得到表 1 所给文法，表中 8 条产生式就是消除左递归的结果。

从结构上看，改写后的 G、S 只出现在产生式的末尾，且 E→TG、T→FS 的右部首符号分别是 T 和 F，而 T、F 都不能推导出 ε，因此不可能出现 E⇒+Eα 或 T⇒+Tα 的推导；G、S 本身又是右递归的，对自上而下分析不构成障碍。所以表 1 的文法可以直接进行 LL（1）分析。

3．FIRST 集与 FOLLOW 集

FIRST(α) 是从 α 出发能够推导出的所有可能的开头终结符组成的集合，若 α 能推导出 ε，则把 ε 也计入 FIRST(α)。FOLLOW(A) 是句型中紧跟非终结符 A 之后出现的终结符组成的集合，若 A 可能位于句子的末尾，则把 # 计入 FOLLOW(A)，并且 # 一定属于开始符号的 FOLLOW 集。

计算时先用三条规则求 FIRST 集：若 X 是终结符，则 FIRST(X)＝{X}；若 X→ε 是产生式，则 ε∈FIRST(X)；若 X→Y1Y2…Yk，则把 FIRST(Y1) 中除 ε 外的符号加入 FIRST(X)，若 Y1 能推出 ε，再依次考察 Y2、Y3……FOLLOW 集则按“把 FIRST(右部符号串) 中除 ε 外的符号加入左部非终结符的 FOLLOW 集”“若某符号是右部最后一个符号或其后各符号都能推出 ε，则把左部的 FOLLOW 集加入该符号的 FOLLOW 集”逐条推进，直到集合不再增大为止。

以 FOLLOW(T) 为例：T 出现在 E→TG 与 G→+TG 的右部，紧跟 T 之后的是 G，故把 FIRST(G) 中除 ε 外的符号 + 加入 FOLLOW(T)；又因 G 能推出 ε，T 之后可能为空，此

时 T 之后出现的符号就是 E 之后出现的符号，所以还要把 FOLLOW(E) 加入 FOLLOW(T)，即 FOLLOW(T)＝{+}∪{),#}＝{+,),#}。其余非终结符的计算结果见表 2。

表 2　各非终结符的 FIRST 集与 FOLLOW 集

非终结符	FIRST 集	FOLLOW 集
E	(，i	)，#
G	+，ε	)，#
T	(，i	+，)，#
S	*，ε	+，)，#
F	(，i	*，+，)，#

4．LL（1）文法的判定

由 FIRST 集与 FOLLOW 集可以求出每条产生式 A→α 的 SELECT 集：若 α 不能推导出 ε，则 SELECT(A→α)＝FIRST(α)；若 α 能推导出 ε，则 SELECT(A→α)＝(FIRST(α)－{ε})∪FOLLOW(A)。SELECT 集的含义是：当栈顶为 A、当前输入符号落在 SELECT(A→α)中时，应当选用产生式 A→α。

一个文法是无回溯的自上而下文法（即 LL（1）文法）的充要条件是：对每个非终结符 A 的任意两条不同的候选产生式 A→α 与 A→β，都有 SELECT(A→α)∩SELECT(A→β)＝∅。这条条件保证了预测分析表中每个表项至多只有一条产生式，分析时不必试探，也就不会回溯。按此条件逐条计算的结果见表 3。

表 3　各产生式的 SELECT 集与 LL（1）判定

产生式	SELECT 集	同一左部的候选式是否相交
E → TG	(，i	唯一产生式
G → +TG	+	与 G→ε 不相交
G → ε	)，#	与 G→+TG 不相交
T → FS	(，i	唯一产生式
S → *FS	*	与 S→ε 不相交
S → ε	+，)，#	与 S→*FS 不相交
F → (E)	(	与 F→i 不相交
F → i	i	与 F→(E) 不相交

由表 3 可见，对同一非终结符而言，两条候选产生式的 SELECT 集两两不相交：G 的两条候选式分别对应加号与“右括号／结束符”，S 的两条候选式分别对应乘号与“加号／右括号／结束符”，F 的两条候选式分别对应左括号与 i。因此该文法满足 LL（1）条件，是 LL（1）文法。另外，该文法没有公共左因子，不必再提取左因子。

5．预测分析表的构造

预测分析表是一个二维表，行对应非终结符，列对应终结符（含 #）。构造方法是：对文法中每条产生式 A→α，求出 SELECT(A→α)，凡 a∈SELECT(A→α)，就把 A→α 填入 M[A, a]；其余表项留空，表示出错。按表 3 的 SELECT 集填表，得到表 4。

表 4　预测分析表 M

非终结符	i	+	*	(	)	#
E	E→TG			E→TG		
G		G→+TG			G→ ε	G→ ε
T	T→FS			T→FS		
S		S→ ε	S→*FS		S→ ε	S→ ε
F	F→i			F→(E)		

表 4 就是程序运行时使用的表，只不过在程序中它被存成二维数组 C[5][6]：行下标依次对应非终结符 E、G、T、S、F（数组 v2），列下标依次对应终结符 i、+、*、(、)、(数组 v1)。表中空白处在程序里用 origin='N'标记，查表时若取到的表项为'N'，就判定为语法错误。产生式右部用字符串存放，ε 产生式用字符''^''表示，按表 4 填表的代码见“二、实验内容”第（5）条。

6．分析程序的设计

程序用结构体 type 表示一条产生式：origin 存放产生式左部的非终结符，array 存放右部字符串，length 存放右部长度。文法对象 e、t、g、g1、s、s1、f、f1 分别对应 8 条产生式，其中 g1、s1 的右部是''^''，表示 ε。分析栈用字符数组 A 模拟，栈顶指针为 top；输入串用字符数组 B 保存，b 指向当前待匹配符号，ch 存放当前输入符号。这些数据结构与 8 条产生式的赋值见“二、实验内容”第（2）条与第（6）条。

程序先从键盘逐个读入符号，边读边检查是否属于终结符集合，遇到 # 结束输入；然后把'#'与'E'压入栈，进入主控循环。主控循环与图 1 的流程一一对应：每轮先弹出栈顶 x，若 x 属于终结符表 v1，则看它是否为结束符 #——是则输出 acc 结束分析，否则与当前输入符号 ch 比较，相同则输入指针后移，不同则报匹配失败；若 x 是非终结符，则以 x 为行、ch 为列查表，表项非空就输出该产生式并把右部逆序压栈（右部为''^''时只退栈不入栈），表项为空则报错终止。两个分支的代码见“二、实验内容”第（6）条。

程序的输出分四列：步骤、分析栈、剩余字符、所用产生式。分析栈从左到右依次为栈底到栈顶，因此栈顶符号位于最右端。由于产生式右部是逆序入栈的，右部最左的符号恰好

位于栈顶，这与最左推导的顺序一致。若某一步执行的是推导，则“所用产生式”一列显示 x→Y1…Yk；若执行的是匹配，则显示“匹 配 a”；ε 产生式在输出中显示为 G->^ 或 S->^。

7．错误处理

程序能查出三类错误：输入串含非法字符时在读入阶段即提示并退出；栈顶为终结符但与当前输入符号不同，说明匹配失败；栈顶为非终结符而预测分析表表项为空，说明按当前文法无法继续推导。后两类都以“出 错”加出错符号的形式输出后终止程序；由于采用“遇错即停”的策略，含多个错误的输入串只能报出第一个。

四、 测试数据及运行结果展示

（一）测试数据一：正确串 i+i*i#

该串共 6 个符号（含结束符 #），对应表达式 i+i*i，是文法的句子，程序输出的分析过程见表 5。表中内容已按列整理，与屏幕输出一致，其中 ε 即程序输出中的 ^。

表 5 正确串 i+i*i# 的分析过程

步骤	分析栈	剩余输入串	所用产生式
0	#E	i+i*i#	E→TG
1	#GT	i+i*i#	T→FS
2	#GSF	i+i*i#	F→i
3	#GSi	i+i*i#	匹配 i
4	#GS	+i*i#	S→ ε
5	#G	+i*i#	G→+TG
6	#GT+	+i*i#	匹配 +
7	#GT	i*i#	T→FS
8	#GSF	i*i#	F→i
9	#GSi	i*i#	匹配 i
10	#GS	*i#	S→*FS
11	#GSF*	*i#	匹配 *
12	#GSF	i#	F→i
13	#GSi	i#	匹配 i
14	#GS	#	S→ ε
15	#G	#	G→ ε
16	#	#	接受（acc）

由表 5 可以看出，程序每一步都取出栈顶符号：第 0 步栈顶是开始符号 E，按 M[E, i] 选用 E→TG，右部 TG 逆序入栈后栈变为 #GT；第 1 步栈顶为 T，按 M[T, i] 选用 T→FS；第 2 步栈顶为 F，按 M[F, i] 选用 F→i；第 3 步栈顶为终结符 i，与输入符号相同，匹

配成功并把输入指针后移到 +。第 4 步栈顶为 S，此时输入符号是 +，M[S, +] 的表项是 S → ε，说明这一层的乘法部分为空，弹出 S 后栈变为 #G。整个过程中栈的变化与最左推导完全对应：

E⇒TG⇒FSG⇒iSG⇒iG⇒i+TG⇒i+FSG⇒i+iSG⇒i+i*FSG⇒i+i*iSG⇒i+i*iG⇒i+i*i，最后栈与输入串都只剩下 #，程序输出 acc!，说明该串被接受。

（二）测试数据二：错误串 i+*i#

该串第 3 个符号是乘号，加号右侧缺一个项，不是文法的句子，分析过程见表 6。

表 6　错误串 i+*i# 的分析过程

步骤	分析栈	剩余输入串	所用产生式
0	#E	i+*i#	E→TG
1	#GT	i+*i#	T→FS
2	#GSF	i+*i#	F→i
3	#GSi	i+*i#	匹配 i
4	#GS	+*i#	S→ ε
5	#G	+*i#	G→+TG
6	#GT+	+*i#	匹配 +
7	#GT	*i#	出错（M[T, *] 为空）

表 6 中前 7 步与正确串的分析过程完全相同，到第 7 步时栈顶为非终结符 T，输入符号为 *，需查 M[T, *]。由表 4 可知该表项为空，说明以 T 开头的符号串不可能以 * 开头，程序因此判定出错并报出“出 错 T”。从语法上看，加号是二元运算符，其右侧必须是一个项，而项的首符号只能是 i 或 (，输入串在 + 之后紧跟 *，缺少了运算对象，所以出错；错误位置是输入串的第 3 个符号，也是分析过程的第 7 步。

（三）运行结果分析

为便于比较，把三次运行的情况汇总于表 7。除题目要求的两个串外，另用带括号的 (i+i)*i# 做了一次补充验证，用以检验 F→(E) 这一产生式以及括号的对消过程是否正确。

表 7　三组输入串的运行结果汇总

输入串	长度（含 #）	分析步数	运行结果	出错位置
i+i*i#	6	17	接受（acc）	—
i+*i#	5	8	出错，栈顶为 T	第 3 个符号 *
(i+i)*i#	8	24	接受（acc）	—

三次运行的结果与手工推导完全一致：两个合法串都在栈与输入串同时只剩 # 时输出 acc，非法串在预测分析表表项为空处准确报错，出错位置与栈顶符号也符合预期。这说明表 4 填写正确，程序中“行取非终结符、列取终结符”的下标映射也正确。补充验证的

(i+i)*i# 共 24 步，含一次 F→(E) 的展开和一次右括号匹配，说明括号嵌套的分析同样正确。

在核对输出时也发现参考程序存在几处实现上的瑕疵，虽不影响本例的分析结论，但值得注意。其一，剩余串的输出用 l = j 作为右边界，而 j 在读入 # 之后已经指向 # 的下一个位置，因此每行末尾会多打印一个未初始化的字符，应改为 l = j - 1。其二，分析栈的输出循环写到 a <= top + 1，它依赖“先出栈再打印”这一顺序才恰好正确，若调整打印位置就会多输出一个字符。其三，分析成功与出错都用 exit(1) 结束，外部无法从返回码区分两种情况。其四，FIRST 集、FOLLOW 集和预测分析表都是硬编码的，换一个文法就必须手工改表和改程序。

对照实验要求还可以看出，实验指导书要求的“手动 / 自动两种方式输入文法”“自动判断是否为 LL（1）文法”“显示 FIRST 集与 FOLLOW 集”这三项，参考程序并未实现，本报告中的 FIRST / FOLLOW 集、SELECT 集和预测分析表是手工推导得到的。若要补齐这些功能，需要把文法产生式读入内存，用迭代法自动计算 FIRST 与 FOLLOW，再按 SELECT 集自动生成分析表，程序的结构将从“表驱动”变为“文法驱动”；同时应采用紧急方式的错误恢复，跳过输入符号直到遇到 FOLLOW 集中的符号，并为栈和指针加边界检查。

结　论

本次实验以算术表达式文法为对象，完成了 LL（1）分析法的全过程实践：消除了文法的左递归，求出 5 个非终结符的 FIRST 集与 FOLLOW 集，按 SELECT 集判定该文法为 LL（1）文法，构造了 5 行 6 列的预测分析表，并用 C++ 实现了表驱动的分析程序。程序对正确串 i+i*i# 输出 17 步分析过程并最终接受，对错误串 i+*i# 在第 7 步报出“出 错 T”，分析栈与剩余输入串的变化与手工推导逐行一致。

实验中的主要难点有三处。一是理解预测分析法的驱动机制：栈顶符号与当前输入符号共同决定下一步动作，产生式右部必须逆序入栈，ε 产生式不能真的压栈（程序中用 '^' 占位后再弹出）；二是理解消除左递归与 LL（1）分析的关系：左递归会使分析陷入 A⇒Aα⇒Aαα⋯ 的死循环，必须先改写文法，改写后引入的 E′、T′ 就是表中的 G、S；三是分清两类出错：终结符不匹配说明括号或运算符不成对，表项为空说明遇到了不该出现的符号。

通过这次实验，我最深的体会是：预测分析法把文法知识编译成一张表，运行时只做查表与栈操作，速度快、结构简单；但表的正确性完全取决于 FIRST / FOLLOW 的推导。